Lab Assignment 4 :

Implement all three practicals in Python (or any suitable programming language) in a single program, and attach screenshots showing the source code and the corresponding output for each practical:

1. Implement the extended Euclidean algorithm.
2. Implement Encryption and decryption using the Additive Cipher
3. Implement Encryption and decryption using the Multiplicative Cipher

Extended Euclidean Algorithm

```
def extended_euclidean(a,b):
        if(b == 0):
            return a,1,0
        gcd,x1,y1 = extended_euclidean(b,a%b)
        x = y1
        y = x1 - (a//b)*y1
        return gcd,x,y
```

Additive Cypher

```
def get_pos(character:str,upper:bool=True):
        return ord(character) - ord('A') if upper else ord(character)-
ord('a')

def add_encrypt(original_string:str,key:int=0)->str :
    if key < 0 or key > 25:
        return "invalid key"
    cyph:str = list(map(lambda x: chr(((get_pos(x)+key)%26) + ord('A')) if
x.isupper() else chr(((get_pos(x,False)+key)%26) +
ord('a')),original_string))
    return "".join(cyph)

def add_decrypt(cyph:str,key:int=0)->str:
    if key < 0 or key > 25:
        return "invalid key"
    cyph:str = list(map(lambda x: chr(((get_pos(x)-key)%26) + ord('A')) if
x.isupper() else chr(((get_pos(x,False)-key)%26) + ord('a')),cyph))
    return "".join(cyph)
```

Multiplicative Cypher

```
def get_pos(character:str,upper:bool=True):
        return ord(character) - ord('A') if upper else ord(character)-
ord('a')
def mul_encrypt(original_string:str,key:int=0)->str :
    if key < 0 or key > 25:
        return "invalid key"
    cyph:str = list(map(lambda x: chr(((get_pos(x)*key)%26) + ord('A')) if
x.isupper() else chr(((get_pos(x,False)*key)%26) +
ord('a')),original_string))
    return "".join(cyph)



def mul_decrypt(cyph:str,key:int=0)->str:
    def modular_inverse(key,base=26):
        gcd,x,y = extended_euclidean(key,base)
        if gcd != 1:
            raise ValueError("modular inverse does not exist")
        return x%base

    if key < 0 or key > 25:
            return "invalid key"
    cyph:str = list(map(lambda x: chr(((get_pos(x)*modular_inverse(key))%26)
+ ord('A')) if x.isupper() else
chr(((get_pos(x,False)*modular_inverse(key))%26) + ord('a')),cyph))
    return "".join(cyph)
```

Complete Program

```
def extended_euclidean(a,b):
        if(b == 0):
            return a,1,0
        gcd,x1,y1 = extended_euclidean(b,a%b)
        x = y1
        y = x1 - (a//b)*y1
        return gcd,x,y

def get_pos(character:str,upper:bool=True):
        return ord(character) - ord('A') if upper else ord(character)-
ord('a')

def add_encrypt(original_string:str,key:int=0)->str :
    if key < 0 or key > 25:
        return "invalid key"
    cyph:str = list(map(lambda x: chr(((get_pos(x)+key)%26) + ord('A')) if
x.isupper() else chr(((get_pos(x,False)+key)%26) +
ord('a')),original_string))
    return "".join(cyph)

def add_decrypt(cyph:str,key:int=0)->str:
    if key < 0 or key > 25:
        return "invalid key"
    cyph:str = list(map(lambda x: chr(((get_pos(x)-key)%26) + ord('A')) if
```

```
x.isupper() else chr(((get_pos(x,False)-key)%26) + ord('a')),cyph))
    return "".join(cyph)


def mul_encrypt(original_string:str,key:int=0)->str :
    if key < 0 or key > 25:
        return "invalid key"
    cyph:str = list(map(lambda x: chr(((get_pos(x)*key)%26) + ord('A')) if
x.isupper() else chr(((get_pos(x,False)*key)%26) +
ord('a')),original_string))
    return "".join(cyph)



def mul_decrypt(cyph:str,key:int=0)->str:
    def modular_inverse(key,base=26):
        gcd,x,y = extended_euclidean(key,base)
        if gcd != 1:
            raise ValueError("modular inverse does not exist")
        return x%base

    if key < 0 or key > 25:
            return "invalid key"
    cyph:str = list(map(lambda x: chr(((get_pos(x)*modular_inverse(key))%26)
+ ord('A')) if x.isupper() else
chr(((get_pos(x,False)*modular_inverse(key))%26) + ord('a')),cyph))
    return "".join(cyph)

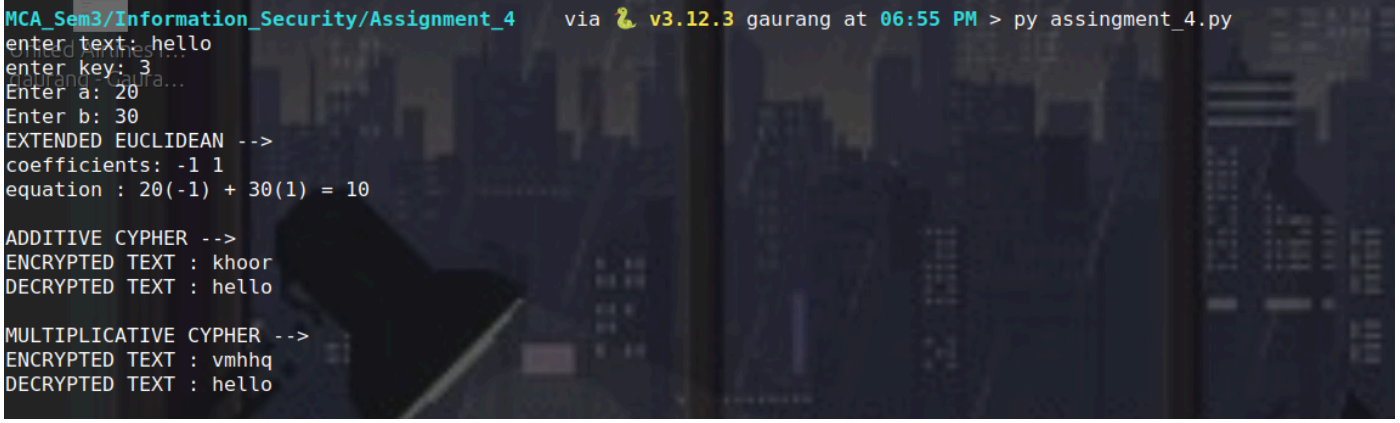
if __name__ == "__main__":
    text:str = input("enter text: ")
    key:int = int(input("enter key: "))
    a:int = int(input("Enter a: "))
    b:int = int(input("Enter b: "))

    gcd,x,y = extended_euclidean(a,b)
    print("EXTENDED EUCLIDEAN -->")
    print(f"coefficients: {x} {y}")
    print(f"equation : {a}({x}) + {b}({y}) = {gcd}\n")

    print("ADDITIVE CYPHER -->")
    cyph:str = add_encrypt(text,key)
    print(f"ENCRYPTED TEXT : {cyph}")
    print(f"DECRYPTED TEXT : {add_decrypt(cyph,key)}\n")

    print("MULTIPLICATIVE CYPHER -->")
    cyph:str = mul_encrypt(text,key)
    print(f"ENCRYPTED TEXT : {cyph}")
    print(f"DECRYPTED TEXT : {mul_decrypt(cyph,key)}\n")
```

OUTPUT


MCA_Sem3/Information_Security/Assignment_4 via v3.12.3 gaurang at 06:55 PM > py assingment_4.py
enter text: hello
enter key: 3
Enter a: 20
Enter b: 30
EXTENDED EUCLIDEAN -->
coefficients: -1 1
equation : 20(-1) + 30(1) = 10
ADDITIVE CYPHER -->
ENCRYPTED TEXT : khoor
DECRYPTED TEXT : hello
MULTIPLICATIVE CYPHER -->
ENCRYPTED TEXT : vmhhq
DECRYPTED TEXT : hello